

UNITÉ 5 · SEMAINE 7

Tubes et composition de processus

📅 Semaine 7 · dates à confirmer </> Laboratoire TD4 (corrigé + défi IA) ~ 2 animations

LABORATOIRE TD4 · CORRIGÉ

Tubes anonymes et composition de processus

Comment le shell construit `ls | grep | wc` : un tube par `|`, un processus par commande, `dup2` pour brancher la sortie de l'une sur l'entrée de la suivante et, surtout, fermer à temps les extrémités inutilisées.



Rapport du TP

Rendez ce TP avec un court rapport en LaTeX, avec le modèle du cours : chaque exercice expliqué, le code qui compte et ce que vous avez exécuté dans le terminal. Vous pouvez utiliser l'IA générative pour écrire le LaTeX : déclarez-le et vérifiez chaque résultat. Modèle, exemple et guide de Prism →

EXERCICES

- | | | | |
|---|---|---|---|
| 1 | Un tube entre parent et enfant | 4 | Un pipeline de commandes fixe |
| 2 | Les règles des tubes : capacité, blocage et SIGPIPE | 5 | Le pipeline depuis la ligne de commande |
| 3 | <code>ps</code> en majuscules | | |

TD4 · 1

● Corrigé · testé

Un tube entre parent et enfant

Énoncé. Créez un tube avec `pipe(2)` et dupliquez le processus. Le parent écrit « Hello World » 10 fois, annonce sur la sortie d'erreur qu'il s'en va et se termine. L'enfant lit au plus 500 octets en une seule lecture, les affiche, l'annonce et se termine. Utilisez des macros pour les constantes.

Solution



C tds/td4/mypipe.c

```

/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a
 * child (reader).
 * ./mypipe [count]      the parent writes "Hello World\n" `count` times
 * (default 10)
 * The child reads AT MOST 500 bytes, once, prints them and leaves.
 *
 * Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the
 * pipe capacity)
 * the parent blocks on write() once the pipe is full. When the child exits,
 * nobody can read
 * any more: the next write() raises SIGPIPE and kills the parent ("Broken
 * pipe", status 141
 * in the shell). That only works if the parent CLOSED its own read end: with
 * -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a
 * potential reader,
 * the kernel never raises SIGPIPE and the parent blocks forever.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE  "Hello World\n"
#define READ_MAX 500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end,
    fds[1] write end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                            /* child: reader */
        exit_if(close(fds[1]) == -1, "close"); /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
    }
}

```

```

    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
    fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
    exit_if(close(fds[0]) == -1, "close");
    return EXIT_SUCCESS;
}

/* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");          /* THE line that
exercise 2 is about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
    fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n",
count);
    exit_if(close(fds[1]) == -1, "close");
    exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    return EXIT_SUCCESS;
}

```

Sortie réelle

```

$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave

```

`fds[0]` est l'extrémité de lecture et `fds[1]` celle d'écriture (comme 0 = entrée et 1 = sortie). Après `fork`, les deux processus détiennent les deux extrémités: chacun ferme celle qu'il n'utilise pas.

Les règles des tubes : capacité, blocage et SIGPIPE

Énoncé. Passez en argument le nombre d'écritures du parent: 50, 100, 500, 1000, 5000, 10000... Si le programme semble bloqué, vérifiez l'état des processus, dessinez les tables de descripteurs et corrigez-le. Avec le programme corrigé, quel est son statut de sortie ?

Solution

Sortie réelle

```
$ python3 -c 'import os, fcntl; r, w = os.pipe(); print(fcntl.fcntl(w,
fcntl.F_GETPIPE_SZ), "bytes")'
65536 bytes
```

Sortie réelle

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 =
killed by timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

Sortie réelle

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"
I am the child, I read 500 bytes and I leave
exit status of mypipe: 141
```

Nouveau : la course, mesurée. L'énoncé d'origine proposait de trouver « à partir de combien de messages » le parent meurt. Sur une machine multicœur, il n'y a pas de seuil fixe : l'enfant lit 500 octets et part pendant que le parent écrit encore, et le gagnant dépend de l'ordonnanceur. 30 exécutions par taille :

Sortie réelle

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses;
done
  50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
 100 messages: 30 x status 0      0 x status 141 (SIGPIPE)
 200 messages: 28 x status 0      2 x status 141 (SIGPIPE)
 300 messages: 19 x status 0     11 x status 141 (SIGPIPE)
 400 messages:  5 x status 0     25 x status 141 (SIGPIPE)
 500 messages:  5 x status 0     25 x status 141 (SIGPIPE)
1000 messages:  0 x status 0     30 x status 141 (SIGPIPE)
```

Le voir animé → A5.2

1. Le tube a une capacité limitée: **65 536 octets** par défaut sous Linux. $10\,000 \times 12$ octets = 120 000 octets ne tiennent pas: quand il est plein, `write` *bloque* le parent jusqu'à ce que quelqu'un lise.
2. L'enfant lit 500 octets et se termine. Si le parent **n'a pas fermé** sa propre extrémité de lecture (`mypipe_bug`), le noyau voit qu'il existe encore un lecteur — le parent lui-même — et le laisse bloqué pour toujours: `timeout` le tue au bout de 3 s (statut 124).
3. Avec l'extrémité fermée, une fois l'enfant parti il n'y a plus de lecteur: le `write` suivant reçoit le signal **SIGPIPE**, dont l'action par défaut est de terminer le processus. Le shell affiche « Broken pipe » et le statut $141 = 128 + 13$.

Animation A5.2 · Capacité du tube →

TD4 · 3

● Corrigé · testé

ps en majuscules

Énoncé. L'enfant lit le tube en continu et affiche en majuscules ce qu'il lit. Le parent redirige sa sortie standard vers le tube avec `dup2` et exécute `ps` avec `exec1p`.

Solution



C tds/td4/upper.c

```

/*
 * TD4 - exercise 3: ps | uppercase, built by hand.
 * The child reads the pipe until end of file and prints it in uppercase.
 * The parent redirects its stdout to the pipe (dup2) and becomes `ps`
 (execlp).
 * The child sees end of file when `ps` exits, because it was the last
 writer.
 */
#include <ctype.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define BUFFER_SIZE 512

static void to_upper(int fd_in, int fd_out)
{
    char buffer[BUFFER_SIZE];
    ssize_t n;

    while ((n = read(fd_in, buffer, sizeof buffer)) > 0) {    /* until end
of file */
        for (ssize_t i = 0; i < n; i++)
            buffer[i] = (char)toupper((unsigned char)buffer[i]);
        exit_if(write_all(fd_out, buffer, (size_t)n) == -1, "write");
    }
    exit_if(n == -1, "read");
}

int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                                           /* child */
        exit_if(close(fds[1]) == -1, "close");              /* otherwise it never sees
end of file */
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
    }
}

```

```

}

/* parent */
exit_if(close(fds[0]) == -1, "close");
exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
exit_if(close(fds[1]) == -1, "close");      /* 1 already refers to the
pipe */
execlp("ps", "ps", (char *)NULL);          /* "ps" twice: file to run,
then argv[0] */
exit_if(1, "execlp ps");
return EXIT_FAILURE;
}

```

Sortie réelle

```

$ ./upper | head -4
  PID TTY          TIME CMD
  6706 ?           00:00:25 UVICORN
  9771 ?           00:00:03 SYSTEMD
  9801 ?           00:00:00 (SD-PAM)

```

L'enfant se termine quand `read` renvoie 0, et cela n'arrive que lorsque *toutes* les extrémités d'écriture sont fermées: celle de `ps` (quand il se termine) et la sienne, c'est pourquoi il la ferme au début. `ps` ne sait pas qu'il écrit dans un tube: son descripteur 1 pointe simplement vers lui.

Un pipeline de commandes fixe

Énoncé. Exécutez le pipeline défini dans une variable `commands` (par exemple `ls -l | tr -d '[:blank:]'`), avec un nombre quelconque de commandes. Le programme se termine quand la dernière se termine et renvoie son statut; si elle est morte d'un signal, il l'indique et renvoie `EXIT_FAILURE`.

Solution



C tds/td4/pipeline.c

```

/*
 * TD4 - exercise 4: run a fixed pipeline, like the shell does for ls -l |
 * tr -d '[:blank:]'
 * Works for any number of commands. Returns the exit status of the LAST
 * command, or
 * EXIT_FAILURE (with a message) if it was killed by a signal.
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

static char *ls[] = { "ls", "-l", NULL };
static char *tr[] = { "tr", "-d", "[:blank:]", NULL };
static char **commands[] = { ls, tr };
static const int ncommands = sizeof commands / sizeof commands[0];

int main(void)
{
    int in = STDIN_FILENO;      /* where the next command reads from */
    pid_t last = -1;

    for (int i = 0; i < ncommands; i++) {
        int fds[2] = { -1, -1 };
        if (i < ncommands - 1) /* every command but
the last writes to a pipe */
            exit_if(pipe(fds) == -1, "pipe");

        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0) {
            if (in != STDIN_FILENO) { /* not the first: read
from previous pipe */
                exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
                exit_if(close(in) == -1, "close");
            }
            if (fds[1] != -1) { /* not the last: write
to the new pipe */
                exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
                exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
            }
        }
    }
}

```

```

        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }

    /* parent: close what the child inherited, keep only the read end for
    the next one */
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
    last = pid;
}

/* Wait for every child (no zombies), remember the status of the last
one. */
int status, last_status = 0;
pid_t pid;
while ((pid = wait(&status)) > 0)
    if (pid == last)
        last_status = status;

if (WIFEXITED(last_status))
    return WEXITSTATUS(last_status);
if (WIFSIGNALED(last_status))
    fprintf(stderr, "pipeline: last command killed by %s\n",
    signame(WTERMSIG(last_status)));
return EXIT_FAILURE;
}

```

Sortie réelle

```

$ cd demo && ../pipeline
total 8
-rw-rw-r--1 student student 4 Sep 30 13:13 one.txt
-rw-rw-r--1 student student 2 Sep 30 13:13 two.txt

```

Le schéma: chaque tour crée le tube *suivant* (sauf pour la dernière commande); l'enfant prend le tube précédent (`in`) comme entrée et le nouveau comme sortie; le parent ferme ce que l'enfant a déjà hérité et garde l'extrémité de lecture pour le tour suivant. À la fin, le parent attend **tous** les enfants (sinon ils resteraient zombies) et garde le statut du dernier.

TD4 · 5

● Corrigé · testé

Le pipeline depuis la ligne de commande

Énoncé. Comme le précédent, mais les commandes viennent de la ligne de commande, séparées par `--` : `pipeline ls -l -- grep a -- wc -l` équivaut à `ls -l | grep a | wc -l`.

Solution

C tds/td4/pipeline_args.c

```
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s cmd [args] [-- cmd [args]]...\n",
argv[0]);
        return EXIT_FAILURE;
    }

    /* Split argv in place. commands[k] points to the first word of command
k. */
    char **commands[argc];
    int ncommands = 0;
    commands[ncommands++] = &argv[1];
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--") == 0) {
            argv[i] = NULL; /* ends the previous
command */
            if (i + 1 ≥ argc || strcmp(argv[i + 1], "--") == 0 ||
commands[ncommands - 1][0] == NULL) {
                fprintf(stderr, "%s: empty command\n", argv[0]);
                return EXIT_FAILURE;
            }
            commands[ncommands++] = &argv[i + 1];
        }
    }
}
```

Sortie réelle

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Sortie réelle

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

Sortie réelle

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Astuce: `argv` est déjà un tableau de chaînes terminé par `NULL`. En remplaçant chaque `--` par `NULL`, chaque commande devient une tranche de `argv` prête pour `execvp`, sans rien copier. Le reste est identique à l'exercice 4.

Liste de vérification

Défi avec l'IA générative · Les tubes dans votre mini-shell

Objectif. Ajoutez l'opérateur `|` au mini-shell en réutilisant `pipeline_args.c`, et diagnostiquez avec l'IA un interblocage provoqué exprès.

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

J'ai un mini-shell et un programme qui exécute des pipelines de N commandes [coller les deux]. (1) Aide-moi à intégrer l'opérateur `|` dans le shell. (2) Ensuite, je supprimerai exprès un `close()` dans le parent : explique, en dessinant les tables de descripteurs de chaque processus, pourquoi ``ls | wc -l`` ne se termine plus, et comment tu le confirmerais avec ``ls -l /proc/<pid>/fd``. Ne corrige pas le code toi-même : guide-moi par des questions.

Comment vérifier le résultat

- ``ls -l | grep c | wc -l`` donne le même résultat que bash.
- Sans le `close()`, la commande se bloque et ``/proc/<pid>/fd`` montre l'extrémité d'écriture encore ouverte.
- Vous savez dire quel processus détenait le descripteur qui empêchait la fin de fichier.

Déclarez votre usage de l'IA : outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.